



ST JOSEPH'S INSTITUTION
INTERNATIONAL SCHOOL

"Enter to Learn, Leave to Serve"

Common Assessment Project Based Learning - Term 1 **Year 9 Computer Science**

Project Deadline: 06/11/2026

Description of assessment

This project will provide the opportunity to students to show what they have learnt in the Algorithmic Thinking and Python Development Units, by creating a series of six programs which will be assessed using the rubric on the last page of this document.

Students will:

Construct six programs to showcase the skills learnt throughout this term of study.

The programs will need to include the following tools, although not every program will need all of them:

- Input function
- Print function
- Variable/s
- If, Elif, Else statements
- For, While loops
- Casting (Integers, Floats)
- Strings
- Assignment statements

NOTE: Comments are expected to be on all six programs to outline specific steps of the program, including inputs, processes (if/else statements, loops), outputs and storage (variables).

Focus of assessment: Developing a set of programs using the Python language

Your task is to use the skills and tools learned in the Algorithmic Thinking and Python Development Units this term to make a series of programs, including Inputs, Outputs, Process and Storage (IPOS) in a python editor of your choice.

1. Create a program (using a for loop) which:
 - Gets the program to ask the user to type in a number between 1 and 100
 - Uses a **for loop** to count **down** from that number to 1.
 - When the countdown finishes, print "**Blast off!**"

For example, if the user enters "5", the program will output the following:

Expected Output

Enter a starting number: 5

5

4

3

2

1

Blast off!

2. Create a program (using if/else statements) which:
 - Gets the program to ask the user for their **age** and whether the movie is during the **day** or **night**.

Ticket prices are based on the user's age and time of the show:

Daytime tickets:

- Child (under 12): \$8
- Teen (13–17): \$10
- Adult (18 and above): \$12

Night tickets:

- Child (under 12): \$10
- Teen (13–17): \$12
- Adult (18 and above): \$15

The program should calculate and display the correct **ticket price**.

Expected Outcome:

Enter your age: 15

Is the movie during the day or night? night

The ticket price is \$12.

3. Create a program (using a while loop) which:

- Create a program (using a while loop) which:
- Stores a random number for the user to guess between 1 and 500
- Asks the user to guess the number the game is "thinking of"
- If the number the user entered is correct, then exit the program by saying "You win!"
- If the number is incorrect, the user will be given 10 tries, and after that, the program will exit by saying "Too many incorrect guesses, the answer is (____)" and output the random number

The program should also give hints: If the guess is within 10 of the secret number, display "Very close! You are too near." If the guess is within 50 of the secret number, display "Not bad, but still far." Otherwise, display "Too far!"

For a challenge, extend the program to allow the user to play again and generate a new random number

Expected Outcome:

I'm thinking of a number between 1 and 500...

Guess the secret number: 120

Too far!

You have 9 guesses left.

Guess the secret number: 480

Very close! You are too near.

Guess the secret number: 489

You got it! You win!

Do you want to play again? (yes/no): no

Thanks for playing! Goodbye

4. A system is required to calculate the delivery cost of parcels. All parcels have a fixed charge for delivery if the parcel is 5kg or below in weight. For local deliveries this charge is \$ 20, for international deliveries the charge rises to \$40.

The maximum weight limit for International deliveries is 5Kg; however, for local deliveries extra weight is permitted and charged at \$ 1 for every 1 kg above that limit.

Create a Python program that will identify if the parcel is local or international and then apply the appropriate weight formula to calculate the correct answer.

Expected Outcome:

Is the parcel local or international? local

Enter the weight of the parcel (in kg): 8

The delivery cost is \$23

Is the parcel local or international? international

Enter the weight of the parcel (in kg): 4

The delivery cost is \$40

Is the parcel local or international? international

Enter the weight of the parcel (in kg): 6

Sorry, the maximum weight limit for international delivery is 5kg.

5. Create a program (using if/else statements) which:

- Simulates the use of a vending machine
- Displays the names of between 6-8 products
- Allows the user to choose to purchase one of the products (this can be indicated using numbers)
- Asks the user to input the amount of money they will use to pay for the product
- Calculates the change required to return to the user
- Or outputs that the amount is not sufficient to pay for the product, requesting the user to input another amount to pay for the product (this new amount will be added to the previous amount entered into the program)
- Outputs the change to the user
- For a challenge you could extend the program to output the amount of change in specific denominations of notes and coins. For example: RM3.25 change will output 3 x 1RM, 1 x 20c and 1 x 5c.

Expected Outcome:

Welcome to the Vending Machine!

1. Milo - RM2.50
2. Pepsi - RM2.20
3. Water - RM1.50
4. Iced Lemon Tea - RM2.80
5. Chocolate Bar - RM3.00
6. Orange Juice - RM2.00
7. Green Tea - RM2.70
8. Coffee - RM3.50

Enter the number of the product you want to buy: 1

You selected Milo - RM2.50

Enter amount to pay: 5

Payment successful! Your change is RM2.50

Change breakdown: 2 x RM1, 1 x 50c

Enjoy your Coke

6. Create a program (using if/else statements) that simulates a parking payment system.

Your program should:

- Display a list of 6–8 vehicle types (e.g., car, motorbike, truck) and their parking fees.
- Allow the user to choose their vehicle type (by entering a number).
- Ask the user to enter the amount of money they are paying.
- If the payment is less than the required parking fee, display a message saying the amount is insufficient and ask the user to add more money (the new amount will be added to the total paid).
- Once the payment is complete, display the vehicle type, total paid, and the change to be returned.

Expected Outcome:

Welcome to the Parking Payment System!

1. Car - RM6.00

2. Motorbike - RM3.00

3. Van - RM10 .00

4. Truck - RM8.00

5. Bus - RM15.00

6. Bicycle - RM1.00

Enter your vehicle type (1-6): 1

You selected Car - RM6.00

Enter amount to pay: 5

Insufficient payment! You still need RM1.00 more.

Enter additional amount: 1

Payment successful!

Vehicle Type: Car

Total Paid: RM6.00

Change: RM0.00

Thank you for using the Parking Payment System!

Suggested Revision Approaches:

- Study the PowerPoint slides provided on Google Classroom
- Refer to the handouts uploaded on Classroom.
- Revise the Python tutorials provided on the Google Classroom.

Grading Rubric (Total: 60 marks)

Program	0–2	3–5	6–8	9–10	Marks Awarded
Program 1	The program does not run or has major syntax/logic errors. Does not use a <code>for</code> loop correctly. Countdown or output missing. Shows limited understanding of loop structure.	The program runs but with logical or syntax errors (e.g. counts upward, misses "Blast off!"). Partial understanding of <code>for</code> loop; output incomplete.	The program runs correctly and performs a countdown. Minor formatting issues or missing input validation. "Blast off!" displays correctly.	The program runs perfectly with no errors. Efficient and correct use of <code>for</code> loop. Clear countdown, correct output, and user-friendly design. Includes validation (e.g. input between 1–100).	/10
Program 2	The program fails to run or contains major syntax/logic errors. Does not use <code>if/else</code> correctly. No clear distinction between age groups or show times. Output incorrect or missing.	Program runs but with logic or syntax errors (e.g., wrong ticket prices or missing conditions). Partial understanding of age/time-based pricing. Output inconsistent.	The program runs correctly with proper age and time checks. Displays correct ticket prices with minor formatting or input issues. Clear use of <code>if/else</code> structure.	The program runs perfectly with no errors. Accurate calculations for all age groups and show times. User-friendly design, well-structured <code>if/else</code> logic, clear prompts, and correct output formatting.	/10
Program 3	The program does not run or has major syntax errors. <code>while</code> loop missing or incorrectly used. No limit on guesses or feedback messages. Incorrect or missing output.	The program runs but has logic errors (e.g., no guess limit, wrong messages, or missing conditions). Partial understanding of the <code>while</code> loop or random number generation.	The program runs correctly with proper random number generation, guess limit, and hints ("too near," "too far"). Minor issues with formatting or user input handling.	Fully functional program with correct random number generation, clear feedback messages, 10-try limit, and replay option. Efficient use of <code>while</code> loop and user-friendly interaction. Accurate, polished output.	/10
Program 4	The program fails to run or has major syntax/logic errors. No correct handling of delivery type or weight conditions. Output mostly incorrect or missing..	Program runs but with logical or syntax errors (e.g., incorrect cost calculation, missing validation for international weight limit). Basic understanding of <code>if/else</code> shown.	The program runs correctly with correct fixed charges and added cost for extra local weight. Handles most conditions accurately with minor output or formatting issues.	Fully functional program with correct logic for local and international deliveries, accurate calculations, and proper validation for weight limits. Code is efficient, user-friendly, and clearly structured.	/10
Program 5	The program fails to run or has major syntax or logic errors. Product selection and payment functions do not work correctly, and outputs are mostly missing or incorrect. Shows little understanding of <code>if/else</code> logic.	The program runs but contains logical or syntax errors such as incorrect price calculations or missing validation. Basic use of <code>if/else</code> is attempted but not fully accurate. User interaction is limited or unclear.	The program runs correctly with most functions working, including product selection, payment handling, and change calculation. User interaction is mostly clear and accurate, with only minor formatting or logical issues.	The program is fully functional, user-friendly, and well-structured. All product and payment options work perfectly with correct change calculation. Advanced features such as showing change in specific denominations are included. Code is efficient, readable, and clearly organized. execution.	/10
Program 6	The program fails to run or contains major syntax or logic errors. Vehicle selection or payment calculation does not work correctly. Output is mostly missing, unclear, or incorrect. Shows little understanding of <code>if/else</code> structures.	The program runs but includes logic or syntax errors such as incorrect fee handling or incomplete payment validation. Basic use of <code>if/else</code> statements is shown, but user prompts and outputs are unclear or inconsistent.	The program runs correctly and handles most required functions, including vehicle selection, payment processing, and change calculation. User input and output are clear and functional, with minor logical or formatting issues.	The program is fully functional, accurate, and user-friendly. It correctly displays all vehicle options, calculates total payment and change, and validates insufficient payments properly. Code is efficient, neatly structured, and may include enhancements such as formatted outputs or clear instructions for users.	/10

